PDF 2 — Engineering & Operations Lessons (DataMate v3, đến 03/11/2026)

Phạm vi: kinh nghiệm lập trình, cấu hình môi trường, quản lý Kaggle, tổ chức thử nghiệm, lỗi đã gặp và cách sửa, bài học thành công / thất bại. Kết quả khoa học ở PDF 1; trạng thái và việc tiếp theo ở PDF 3; cấu hình máy ở `environment_summary.pdf`.

1. Lập trình

1.1 Thiết kế đã chứng minh hiệu quả

- **Ngữ nghĩa nằm trong code đã kiểm chứng** (`src/dm.py`), LLM chỉ viết chương trình rất ngắn. Mỗi định nghĩa rulebook là một hàm có docstring + test; `CANON_DOC` liệt kê cột / hàm cho prompt.
- **Mỗi sửa lỗi là một quy tắc tổng quát + một test** (`tests/test_mapping_fixes.py` mục 12–22). Không logic theo id DN / id câu (user cấm).
- **Hai kiểm chứng bắt buộc sau mỗi thay đổi toolkit / luật**: oracle train (`tests/eval_lib.py`, phải giữ 2967 / 1 / 32) và không báo động giả trên ánh xạ tham chiếu (`tests/test_plausibility.py`, `0 flagged`).
- **Lưu đầu ra từng câu** (chương trình, đáp án, phiếu, ánh xạ ứng viên) → phân tích và bỏ phiếu lại offline trong vài giây (`src/replay.py`, `src/vote.py`, `automation/map_penalties.py`).

1.2 Lỗi lập trình đã gặp (và cách tránh)

Lỗi	Hậu quả	Sửa / bài học
Hàm trùng tên trong class (`weekday_avg` định nghĩa 2 lần)	bản sau ghi đè bản đúng	grep tên hàm trước khi thêm
Regex không dấu "dem" khớp cả "đếm" lẫn "đem về"	luật đếm chặn câu doanh thu	kiểm từ có dấu trên câu gốc khi mơ hồ
"% so" khớp "% số khách"; "lọ" vào khối lô hàng	luật bắn nhầm	neo regex chặt (`% so (voi|vs|cung)`)
`df[c].dtype == object` bỏ sót cột `string` của pandas 3	kiểm tra không chạy	dùng `pd.api.types.is_string_dtype`
Kiểm tra doanh thu = tổng dòng báo nhầm DN chỉ có đơn	báo động giả biz_110	chỉ áp dụng khi dòng có mặt hàng
`_Timeout` thoát khỏi worker `imap_unordered`	chết 2/4 shard	bắt ngoại lệ trong worker
`make_kernel` chỉ thay token `OUT` đứng riêng	`OUT/train` ghi vào thư mục bị xoá	thay theo tiền tố
`agent.py` chạy dưới `__main__`	pickle / import hỏng	test CLI dưới `__main__`
Sửa file bằng script Python với chuỗi `'''` lồng nhau	SyntaxError, không sửa gì	viết script ra file riêng, hoặc dùng Edit
Edit bỏ mất dòng comment ngay sau chuỗi cũ	mất chú thích	so diff sau mỗi edit lớn

1.3 Mô hình / vLLM

- Thinking **không greedy** (temperature 0.6, top-k 20 theo model card Qwen3.5); greedy dễ lặp.
- **Budget forcing**: trần suy nghĩ 12288 token; quá trần → cắt vòng lặp, chèn kết thúc suy nghĩ, tiếp tục cùng request.
- FP8 KV cache và `VLLM_BATCH_INVARIANT=1` **không chạy** với Qwen3.5 / vLLM 0.32; MTP chỉ nhanh 6 % → mặc định tắt (flag `--kv-dtype` có fallback theo thứ tự).
- KV thật đọc từ dòng `[engine] ... KV cache N tokens` (≈201k token / L4 ở `--gpu-util 0.92`).

2. Môi trường container (cloud Claude Code)

- Container 4 nhân, 15 GB RAM, không GPU, **mất khi sang khung chat mới** → mọi thứ cần giữ phải commit / push; token Kaggle phải xin lại.

- **Đĩa có hạn mức riêng** (`df` gây hiểu lầm). Hai lần đầy đĩa (08:24, 12:38): thư mục tạm `/tmp/tmp.*`, `/tmp/preflight_*` của test / preflight bị ngắt giữa chừng dồn **22 GB**. Đã sửa: `tests/run_all.sh` (trap EXIT/TERM) và `automation/preflight.py` (atexit + SIGTERM) tự dọn. Xoá `/tmp` hàng loạt cần user cho phép (bị chặn tự động).
- Khi đĩa đầy, lệnh đang chạy chết âm thầm và **file đang sửa có thể không được ghi** → kiểm `git diff` sau sự cố.
- **Không** `pkill -f <m□u>` **/ không lọc** `ps | grep` **theo mẫu có trong chính lệnh** → tự giết shell (exit 144, gặp 3 lần). Cách đúng: liệt kê PID ra file, xem, rồi `kill <PID>` ở lệnh sau.
- Tác vụ nền: chỉ dùng `run_in_background` (timeout 7200000 ms). `cmd &` trong một lệnh thường không báo khi xong.
- Shell chặn `sleep` dài nối lệnh → chờ điều kiện bằng vòng `until …; do sleep 15; done` chạy nền.

3. Kaggle

3.1 Vận hành chuẩn

- `automation/fanout.py <runs/vNN> zc-vNN <jobs> 4 --accounts A,B --l4 --ks …`: tự chạy preflight (từ chối nếu chưa qua), tạo kernel, đẩy, failover khi push lỗi, kéo kết quả. Dồn 2 phiên GPU của một acc rồi mới sang acc khác.
- Theo dõi bằng `automation/watch_done.py <runs> --max 115` (chạy nền); `runs/active_runs.txt` chọn run được theo dõi; `.reported` ghi kernel đã báo (xoá dòng khi đẩy lại cùng slug).
- Một bản đầy đủ = 4 kernel (train + 3 public seed 0/1/2) × ~1.1–2.3 h ≈ **8 giờ GPU**; quota tuần 30 h / acc.

3.2 Lỗi Kaggle đã gặp

Lỗi	Cách xử lý
`Maximum weekly GPU quota of 30.00 hours reached` (caotienminh1, 03/11 14:02)	chuyển acc; ghi lại acc hết quota
`Maximum batch GPU session count of 2 reached`	acc đang bận 2 phiên → acc khác; failover của fanout chỉ thử 1 acc trong danh sách
Đẩy lại tay bằng `make_kernel.py` với file jobs nhiều dòng	file jobs là một dòng ngăn `;` → tách đúng từng job trước khi đẩy (suýt đẩy 4 job vào 1 kernel)
Code dataset `<acc>/zc-code` chưa "ready"	chờ `kaggle datasets status` = ready rồi mới chạy `run_kernel.py`
Import thư viện thẳng từ mount `/kaggle/input`	> 900 s → chết 5 kernel; luôn copy / cài vào môi trường kernel
API 503 đọc thành trạng thái `error`	`run_kernel.py --attach` nối lại
Acc dùng chung với dự án khác (`e27*`, `e32*`, `nclt-*`)	chỉ đụng kernel `zc-*`

4. Tổ chức thử nghiệm

- **Cổng trước khi chạy (user chốt):** tự review diff → `tests/run_all.sh` → `automation/preflight.py` (flag, file kernel, test CPU, chạy thử end-to-end trên CPU với mô hình giả). Hash preflight phủ `src/`, `automation/*.py`, `tests/` (docs không ảnh hưởng). **Mỗi lần sửa code sau preflight phải chạy lại** (~10–12 phút); dồn sửa rồi mới preflight.
- **Thay đổi ở phần chỉ chạy trên Kaggle** (vLLM, môi trường) → chạy nhỏ `--subset` trước.
- Mỗi bản ghi vào `EXPERIMENT_LOG.md`: giả thuyết, cấu hình, số liệu, phân tích, quyết định; kết quả âm cũng ghi.
- **File nộp:** `src/vote.py` (3 lần chạy) → `automation/make_submission.py` (kiểm định dạng) → `submissions/vNN/` + README → **push ngay** và gửi user (một lần quên push v15, user không tìm thấy file).

5. Bài học chiến lược

Thành công - Phân tích bất đồng **không cần nhãn** tìm ra lỗi thật: luật vs LLM (55 câu public), ánh xạ hạng 1 vs hạng 2, đáp án 0 bất thường, so README với bảng chuẩn. Mỗi lần dẫn tới một quy tắc tổng quát. - Đo GPU bằng `gpu_monitor.csv` / `gpu_summary.json`: bận 84–94 %, 4 GPU kết thúc lệch nhau ≤ 20 % nhờ work stealing.

Thất bại / cần tránh - Tin điểm train để chọn bản nộp: train 0.991 (v20) nhưng public thấp hơn v19. Đánh giá bản mới bằng **so sánh câu-theo-câu với bản đã có điểm portal** (giống / đổi / thêm abstain) trước khi nộp. - Thay đổi tăng abstain ở DN lớn (dev_003 129 câu) có thể mất nhiều hơn được; kiểm số câu chuyển sang abstain. - Sửa nhiều thứ liền nhau sau preflight làm preflight phải chạy lại 4–5 lần (mất ~1 giờ): **chốt đợt sửa trước, preflight một lần**.